

In [128...

```
from sklearn.preprocessing import StandardScaler, MaxAbsScaler, MinMaxScaler, Normalizer  
from sklearn.preprocessing import StandardScaler, MaxAbsScaler, MinMaxScaler, Normalizer
```

1. Normalization (Row-wise scaling) Normalizer

Scales each sample (row) to unit norm

Common in text mining, cosine similarity

Norm options:

norm='l1'

norm='l2'

norm='max'

```
from sklearn.preprocessing import Normalizer
```

2. Standardization (Mean–Variance Scaling) StandardScaler

Mean = 0, Standard deviation = 1

Most common scaler

```
from sklearn.preprocessing import StandardScaler
```

Min–Max Scaling MinMaxScaler

Scales features to a given range (default 0–1)

```
from sklearn.preprocessing import MinMaxScaler
```

4. Max Absolute Scaling MaxAbsScaler

Scales by maximum absolute value

Preserves sparsity (good for sparse matrices)

```
from sklearn.preprocessing import MaxAbsScaler
```

5. Robust Scaling (Outlier-Resistant) RobustScaler

Uses median and IQR

Robust to outliers

```
from sklearn.preprocessing import RobustScaler
```

6. Power Transformations (Make Data More Gaussian) PowerTransformer

Methods:

Yeo–Johnson (handles zero & negative values)

Box–Cox (positive values only)

```
from sklearn.preprocessing import PowerTransformer
```

7. Quantile-Based Scaling QuantileTransformer

Maps data to:

Uniform distribution

Normal distribution

Very effective for skewed data

```
from sklearn.preprocessing import QuantileTransformer
```

8. Unit Variance Scaling (No Centering) scale() (function)

Standardizes features but not a transformer

Less used in pipelines

```
from sklearn.preprocessing import scale
```

9. Log / Function-Based Scaling FunctionTransformer

Apply custom transformations:

log

sqrt

exp

```
from sklearn.preprocessing import FunctionTransformer
```

10. Binarization (Threshold Scaling) Binarizer

Converts values to 0 or 1 based on threshold

```
from sklearn.preprocessing import Binarizer
```

Most Used in Practice

StandardScaler

MinMaxScaler

RobustScaler

Normalizer

PowerTransformer

StandardScaler (Z-score normalization)

Use when:

Data is approximately normally distributed

You use:

Linear Regression

Logistic Regression

SVM

PCA

K-Means

Avoid when:

Data has strong outliers

 Most common default choice

MinMaxScaler (0–1 scaling)

Use when:

You need a fixed range

You use:

Neural Networks

Gradient-based algorithms

You want to preserve zero values

Avoid when:

Data contains outliers

3 MaxAbsScaler

Use when:

Data is sparse (many zeros)

You want to preserve sparsity

Typical in:

Text data

TF-IDF matrices

Avoid when:

Dense data with large outliers

4 RobustScaler

Use when:

Data has many outliers

Features are skewed

You use:

Linear models

Clustering with noisy data

Uses median & IQR

5 Normalizer (L1 / L2 / Max)

Use when:

Direction matters more than magnitude

You compare vectors using:

Cosine similarity

Common in:

NLP

Recommendation systems

Note:

Works row-wise, not feature-wise

6 PowerTransformer (Box-Cox / Yeo-Johnson)

Use when:

Data is highly skewed

You want to make features more Gaussian

Helpful before:

Linear regression

PCA

Constraints:

Box-Cox → positive values only

Yeo-Johnson → allows negatives

7 QuantileTransformer

Use when:

Data is non-normal

Many outliers

You want:

Uniform distribution

Normal distribution

Best for:

Tree-based models

Strongly skewed features

8 FunctionTransformer (log, sqrt, etc.)

Use when:

You need custom transformations

Examples:

$\log(x+1)$ for exponential growth

$\sqrt{x}$ for count data

Requires domain knowledge

Binarizer

Use when:

Feature is binary in nature

Threshold-based decisions

Example:

Purchased / Not purchased

```
In [5]: import numpy as np
import pandas as pd
df = pd.read_csv('Data.csv')
df.isnull().sum()
df.dropna(inplace=True)
```

```
In [64]: import numpy as np
import pandas as pd
df = pd.read_csv('Data.csv')
df.head(2)
df.isnull().sum()
df.dropna(inplace=True)
df.head(1)
```

```
Out[64]:
```

	Country	Age	Salary	Purchased
0	France	44.0	72000.0	No

```
In [99]: X = df.iloc[:, 1:3].values
X.shape
y = df.iloc[:, 3:4].values
from sklearn.preprocessing import Normalizer, normalize, scale, QuantileTransformer
from sklearn.preprocessing import Binarizer, StandardScaler, MaxAbsScaler, MinMaxScaler
#X = normalize(X, norm='L2')
#X = scale(X, axis=0)
#X = FunctionTransformer(func=np.log1p).fit_transform(X)
#X = RobustScaler().fit_transform(X)
X = MaxAbsScaler().fit_transform(X)
X
```

```
Out[99]: array([[8.80000000e-01, 8.67469880e-01],
                [5.40000000e-01, 5.78313253e-01],
                [8.00000000e-01, 1.20481928e-04],
                [1.00000000e+00, 6.26506024e-01],
                [9.60000000e-01, 9.51807229e-01],
                [1.00000000e+00, 1.00000000e+00],
                [8.80000000e-01, 8.67469880e-01],
                [8.80000000e-01, 8.67469880e-01],
                [8.80000000e-01, 8.67469880e-01],
                [8.80000000e-01, 8.67469880e-01]])
```

```
In [27]: from sklearn.preprocessing import Normalizer, scale, RobustScaler, Binarizer
from sklearn.preprocessing import FunctionTransformer, QuantileTransformer, PowerTr
X = df.iloc[:, 1:3].values
X = Normalizer().fit_transform(X) # {'L1', 'L2', 'max'}, default='L2'
X = scale(X)
X = Binarizer(threshold=1.0).fit_transform(X)
X = FunctionTransformer(func=np.log1p).fit_transform(X)
X = QuantileTransformer().fit_transform(X)
X = PowerTransformer().fit_transform(X)
X
```

C:\Users\Original\anaconda3\Lib\site-packages\sklearn\preprocessing_data.py:2829: UserWarning: n_quantiles (1000) is greater than the total number of samples (10). n_quantiles is set to n_samples.

```
warnings.warn(
```

```
Out[27]: array([[ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [  3.          ,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ],
                [ -0.33333333,  0.          ]])
```

What is an Outlier?

An outlier is a data point that is very different from most other values. It can be:

Much larger

Much smaller

Simple Example

Heights of people (cm):

[160, 162, 159, 161, 163, 250]

👉 250 cm is an outlier (most values are around 160)

```
In [ ]: Why Outliers Are a Problem

Outliers can:

Distort mean

Reduce model accuracy

Mislead statistical analysis

Example:

Mean without outlier ≈ 161
Mean with outlier ≈ 176
```

How to Detect Outliers (Methods)

Method 1: IQR (Most Common) Lower= $Q1 - 1.5 \times IQR$ Upper= $Q3 + 1.5 \times IQR$

```
In [30]: import numpy as np

data = np.array([160, 162, 159, 161, 163, 250])

Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = data[(data < lower) | (data > upper)]
print(outliers)
```

[250]

Method 2: Z-Score (Normal Data)

$|z| > 3 \Rightarrow \text{outlier}$

```
In [118... from scipy.stats import zscore
import numpy as np

data = np.array([160, 162, 159, 161, 163, 250, 200, 1000, 2000, 3000, 500, 400 ])
z = zscore(data)

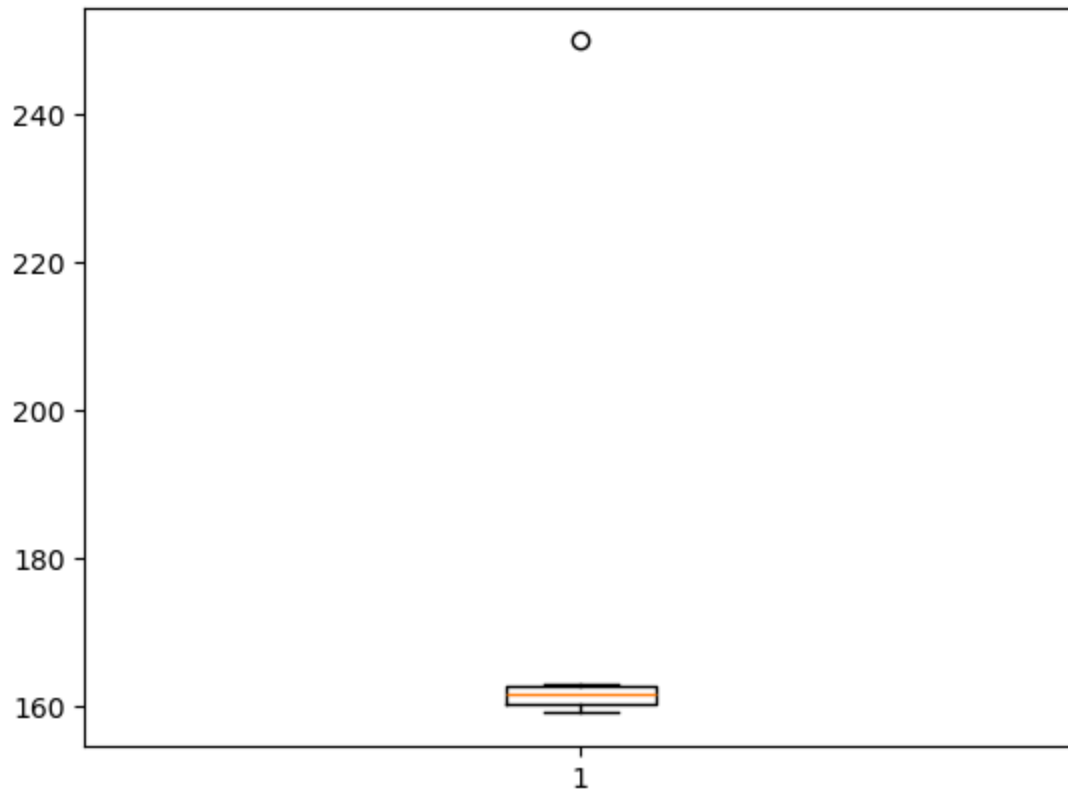
outliers = data[z > 0.3]
print(outliers)
```

[1000 2000 3000]

Method 3: Visualization (Boxplot)

```
In [33]: import matplotlib.pyplot as plt

plt.boxplot(data)
plt.show()
```



```
In [36]: clean_data = data[(data >= lower) & (data <= upper)]
clean_data
```

```
Out[36]: array([160, 162, 159, 161, 163])
```

```
In [37]: import numpy as np

data_capped = np.clip(data, lower, upper)
data_capped
```

```
Out[37]: array([160. , 162. , 159. , 161. , 163. , 166.5])
```

```
In [39]: from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
data_scaled = scaler.fit_transform(data.reshape(-1, 1))
data_scaled
```

```
Out[39]: array([[ -0.6],
               [  0.2],
               [-1.  ],
               [-0.2],
               [  0.6],
               [35.4]])
```

```
In [41]: import numpy as np

data_log = np.log(data)
data_log
```

```
Out[41]: array([5.07517382, 5.08759634, 5.0689042 , 5.08140436, 5.0937502 ,
                5.52146092])
```

When NOT to Remove Outliers

If outliers represent:

Fraud

Rare events

Medical anomalies

Feature Encoding Methods in sklearn

Feature encoding = converting categorical data into numerical form for ML models.

1 Ordinal Encoding OrdinalEncoder

Assigns integers to categories

Keeps order (if it exists)

```
from sklearn.preprocessing import OrdinalEncoder
```

Use when:

Categories have natural order (Low < Medium < High)

⚠ Not suitable for unordered categories

2 One-Hot Encoding OneHotEncoder

Creates binary columns (0/1)

Most widely used

from sklearn.preprocessing import OneHotEncoder

Use when:

No order in categories

Linear models, tree models

3 Label Encoding (Limited Use) LabelEncoder

Encodes labels (target variable)

Not recommended for input features

from sklearn.preprocessing import LabelEncoder

Use when:

Encoding target variable (y) only

```
In [58]: from sklearn.preprocessing import LabelEncoder, LabelBinarizer, OneHotEncoder, OrdinalEncoder
cat = df.iloc[:, 0:1].values
#cat = LabelEncoder().fit_transform(cat)
#cat = OrdinalEncoder().fit_transform(cat)
#cat = LabelBinarizer().fit_transform(cat)
cat
```

```
Out[58]: array([[ 'France'],
                [ 'Spain'],
                [ 'Germany'],
                [ 'Spain'],
                [ 'France'],
                [ 'Germany'],
                [ 'France'],
                [ 'France'],
                [ 'France'],
                [ 'France']], dtype=object)
```

```
In [122... from sklearn.preprocessing import LabelEncoder, OrdinalEncoder, OneHotEncoder, LabelBinarizer
co = df.iloc[:, 0:1].values
co = LabelBinarizer().fit_transform(df[:, 0])
co
```

```
Out[122... array([[1, 0, 0],
          [0, 0, 1],
          [0, 1, 0],
          [0, 0, 1],
          [1, 0, 0],
          [0, 1, 0],
          [1, 0, 0],
          [1, 0, 0],
          [1, 0, 0],
          [1, 0, 0]])
```

```
In [59]: from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
d = ColumnTransformer(transformers=[('encoder', OneHotEncoder(), [0])], remainder='
cat = d.fit_transform(cat)
cat
```

```
Out[59]: array([[1., 0., 0.],
          [0., 0., 1.],
          [0., 1., 0.],
          [0., 0., 1.],
          [1., 0., 0.],
          [0., 1., 0.],
          [1., 0., 0.],
          [1., 0., 0.],
          [1., 0., 0.],
          [1., 0., 0.]])
```

Fix Inconsistent Data

```
In [ ]: # Fix Typos (Categorical Values)
#Problem

#Different spellings for the same value:

#Male, male, M, m

#Solution (Standardize text)
df['gender'] = df['gender'].str.lower().str.strip()

df['gender'] = df['gender'].replace({
    'm': 'male',
    'f': 'female'
})
```

Fix Units (Numerical Values)

Problem

Same feature in different units:

Height: cm and meters

```
In [ ]: df['height_cm'] = df['height'].apply(  
        lambda x: x * 100 if x < 3 else x  
    )
```

Fix Formats (Dates, Numbers, Text)

```
In [ ]: df['date'] = pd.to_datetime(df['date'], errors='coerce')
```

Number Format Inconsistency

Problem

"1,200" "\$500"

```
In [ ]: df['price'] = (  
    df['price']  
    .str.replace(',', '')  
    .str.replace('$', '')  
    .astype(float)  
    )
```

Text Case Inconsistency

```
In [ ]: df['city'] = df['city'].str.title()
```

```
In [127... x = 'ahmeD mohamEd'  
x.title()
```

```
Out[127... 'Ahmed Mohamed'
```

```
In [ ]:
```